# 山东大学 计算机科学与技术 学院

# 自然语言处理 课程实验报告

| 学号:202200130077 | 姓名：薛润泽 | 班级：智能 22 |
|---|---|---|
| 实验题目： 实验一：语言模型 | | |

实验内容：
理解并实践语言模型
1.用 python 编程实践语言模型（uni-gram 和 bi-gram）,加入平滑技术。
2.计算 test.txt 中句子的 PPL，对比 uni-gram 和 bi-gram 语言模型效果。

实验过程中遇到和解决的问题：
（记录实验过程中遇到的问题，以及解决过程和实验结果。可以适当配以关键代码辅助说明，但不要大段贴代码。）
遇到的问题：在计算概率时，如果使用加一平滑，uni-gram 和 bi-gram 的分母用什么？
经过查阅资料,发现不论是 uni-gram 还是 bi-gram 分母都是用在训练集时出现的所有词语的个数+所有去重词语的个数，即 total_account + V。这里考虑的应该是理想情况下，分子代表的词语在训练集中都出现过，按照这个公式仍然可以使得词语总概率为 1。

实验步骤：
1、理解实验模型，uni-gram 与 bi-gram：
Uni-gram 是一元语言模型，假设每个词的出现是完全独立的，与上下文无关。
Bi-gram 模型是二元语言模型，假设每个词的出现仅依赖于前一个词，即当前词的概率由前一个词决定。
模型输入：
训练输入-> 语句集，用来寻找每个词在语句中的出现规律
测试输入-> 语句集，希望计算该语句集的总 Perplexity
模型输出：
测试输出语句集的总 Perplexity

2、使用 train_LM.txt 进行模型语句的训练：
对于 Uni-gram 模型，统计 train_LM.txt 中所有出现词语的出现频率。
对于 Bi-gram 模型，统计 train_LM.txt 中所有出现词语的出现频率的同时，记录每一个 i-1 词语和 i 词语同时出现的频率。

3、使用 test_LM.txt 进行模型测试：

$$PP(W) = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(w_i \mid w_1, \ldots, w_{i-1})\right)$$

计算 Perplexity 的公式如上，其中 N 是语句集中所有词语出现频率的总和。由于在测试语句中，可能会出现训练集中没有出现过的词语，所以对词语的出现频率进行+1 平滑处理，处理公式如下：

$$P(\text{word}) = \frac{\text{count}(\text{word}) + 1}{\text{total_count} + |V|}$$

实验结果：

```
PS D:\大学\大三下\自然语言处理\实验\exp1> python -u "d:\大
学\大三下\自然语言处理\实验\exp1\ex1.py"
Unigram perplexity: 343.05549968287403
Bigram perplexity: 354.69424101885534
PS D:\大学\大三下\自然语言处理\实验\exp1>
```

实验结果得到 Unigram perplexity 约为 343.055，Bigram perplexity 结果约为 354.694

实验总结：

本实验尝试了 uni-gram 与 bi-gram 两个简单的语言模型，尝试通过训练模型，计算测试集的困惑度。初步认识了自然语言处理的任务。

附源代码

```
import os
import math
import nltk
from nltk.tokenize import word_tokenize
from collections import defaultdict, Counter
```

```
START_TOKEN = "<s>"
END_TOKEN = "</s>"

def read_data(file_path): # 把含有__eou__的句集合分成多个单独句子
    sentences = []
    with open(file_path, 'r', encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            if line:
                parts = [part.strip() for part in line.split("__eou__") if
part.strip()]
                sentences.extend(parts)
    return sentences

def preprocess_sentence(sentence):
    tokens = word_tokenize(sentence) # 把一个句子分割成多个词
    return [START_TOKEN] + tokens + [END_TOKEN]

class UnigramLM:
    def __init__(self):
        self.unigram_counts = Counter() #记录每个词出现频率
        self.total_count = 0
        self.vocab = set()

    def train(self, sentences):
        for sentence in sentences: #update 函数可以将 tokens(可迭代操作)中的
每个词都拿出来进行操作
            tokens = preprocess_sentence(sentence)
            self.unigram_counts.update(tokens)  # 每个词的频率
            self.vocab.update(tokens) # 每个词插入集合
        self.total_count = sum(self.unigram_counts.values()) # 记录所有单
词总个数（不去重）

    def prob(self, word):
        #使用加一平滑计算概率：P(word) = (count(word) + 1) / (total_count +
|V|)
        return (self.unigram_counts[word] + 1) / (self.total_count +
len(self.vocab))

    def perplexity(self, sentences):
        total_log_prob = 0
        total_words = 0
        for sentence in sentences:
            tokens = preprocess_sentence(sentence)
```

```
            total_words += len(tokens)
            for token in tokens:
                total_log_prob += math.log(self.prob(token))
        avg_log_prob = total_log_prob / total_words
        return math.exp(-avg_log_prob)

class BigramLM:
    def __init__(self):
        # bigram_counts: 前一个词 -> 后一个词的计数，即两个词同时出现的概率
        self.bigram_counts = defaultdict(Counter)
        # unigram_counts: 统计前一个词的出现次数
        self.unigram_counts = Counter()
        self.vocab = set()

    def train(self, sentences):
        for sentence in sentences:
            tokens = preprocess_sentence(sentence)
            self.vocab.update(tokens)
            self.unigram_counts.update(tokens)
            for i in range(1, len(tokens)):
                prev_word = tokens[i-1]
                curr_word = tokens[i]
                self.bigram_counts[prev_word][curr_word] += 1

    def prob(self, prev_word, word):
        return (self.bigram_counts[prev_word][word] + 1) /
(self.unigram_counts[prev_word] + len(self.vocab))

    def perplexity(self, sentences):
        total_log_prob = 0
        total_words = 0
        for sentence in sentences:
            tokens = preprocess_sentence(sentence)
            total_words += (len(tokens) - 1)  # bigram 从第 1 个 bigram 开始
计算
            for i in range(1, len(tokens)):
                total_log_prob += math.log(self.prob(tokens[i-1],
tokens[i]))
        avg_log_prob = total_log_prob / total_words
        return math.exp(-avg_log_prob)

if __name__ == '__main__':
    train_file = 'train_LM.txt'
    test_file = 'test_LM.txt'
```

```
train_sentences = read_data(train_file)
test_sentences = read_data(test_file)

uni_model = UnigramLM()
uni_model.train(train_sentences)

bi_model = BigramLM()
bi_model.train(train_sentences)

uni_ppl = uni_model.perplexity(test_sentences)
bi_ppl = bi_model.perplexity(test_sentences)

print("Unigram perplexity:", uni_ppl)
print("Bigram perplexity:", bi_ppl)
```